Bus Structures – Memory Locations and Addresses – Instruction and Instruction Sequencing

1. Bus Structures

Definition

A **bus** is a communication pathway that transfers data, addresses, and control signals between components of a computer such as CPU, memory, and I/O devices.

Types of Buses

(a) Data Bus

- Transfers actual data between CPU, memory, and peripherals.
- Bidirectional.
- Width (8, 16, 32, 64 bits) determines how much data moves at once.
- Wider bus → higher performance.

(b) Address Bus

- Carries the address of the memory location to be accessed.
- Unidirectional (CPU → memory/I/O).
- Size determines maximum memory capacity.

Example:
If address bus = n bits → memory locations = 2^n

(c) Control Bus

- Carries control and timing signals.

- Coordinates operations.

Common control signals:

- Read
- Write
- Interrupt
- Clock
- Reset

System Bus Structure

The **system bus** connects:

- CPU
- Main Memory
- Input/Output devices

Functions:

- Data transfer
- Address identification
- Operation control

Single Bus vs Multiple Bus

Feature	Single Bus	Multiple Bus
Cost	Low	High

Feature	Single Bus	Multiple Bus
Speed	Slower	Faster
Complexity	Simple	Complex
Traffic	More congestion	Less congestion

2. Memory Locations and Addresses

Memory Organization

Memory consists of a large number of **storage locations**, each having:

- Unique address
- Fixed size (usually 8 bits = 1 byte)

Memory Location

A **memory location** is a storage cell that holds one unit of data.

Example:

Address	Data
2000H	45H
2001H	A3H

Each location is uniquely identifiable.

Memory Address

A **memory address** is a binary number used by the CPU to access a specific memory location.

Address Space

The total number of unique memory addresses available.

Formula:

If address bus = n bits
→ Address space = 2^n locations

Example:

- 16-bit address bus → 2^{16} = 65,536 locations
- If each location = 1 byte → 64 KB memory

Word Address vs Byte Address

Byte Addressable Memory

- Each byte has a unique address
- Most modern systems use this

Word Addressable Memory

- Address refers to a word (multiple bytes)
- Used in some older architectures

Memory Access Methods

1. Sequential access (tape)
2. Direct access (disk)
3. Random access (RAM) — most important

(I). Sequential Access (Tape)

Definition

In **sequential access**, data is accessed in a fixed linear order. To reach a particular record, all preceding records must be read.

Working Principle

- Data is stored one after another.
- Access begins from the start.
- The system moves sequentially until the desired data is found.

Example

Magnetic tape.

(II). Direct Access (Disk)

Definition

In **direct access**, data can be accessed directly by specifying its address, but access time varies depending on the physical location.

Working Principle

- Storage divided into blocks/sectors
- Read/write head moves to required position
- Data accessed without reading all previous data

Example

Magnetic disk, hard disk.

(III). Random Access (RAM) — Most Important

Definition

In **random access**, any memory location can be accessed directly in equal time, regardless of its position.

Working Principle

- Memory is addressable
- CPU sends address
- Data retrieved instantly
- No mechanical movement

Example

Main memory (RAM), cache memory.

3. Instructions

Definition

An **instruction** is a binary command that tells the CPU to perform a specific operation.

Basic Instruction Format

An instruction typically contains:

- Opcode (operation code)
- Operand (data or address)

General format:

Instruction = Opcode + Operand

Example:

ADD R1, R2

- Opcode → ADD
- Operands → R1, R2

Types of Instructions

1. Data transfer instructions
    - MOV, LOAD, STORE
2. Arithmetic instructions
    - ADD, SUB, MUL, DIV
3. Logical instructions

    - AND, OR, NOT, XOR
4. Control instructions
    - JMP, CALL, RETURN
5. Input/Output instructions

4. Instruction Sequencing

Definition

Instruction sequencing is the process of determining the order in which instructions are executed by the CPU.

Normally, instructions are executed **sequentially**, but control instructions can change the flow.

Program Counter (PC)

- Holds address of next instruction
- Automatically increments after each fetch
- Updated during branch/jump instructions

Basic Instruction Cycle

The CPU performs instructions in cycles.

Step 1: Fetch

- PC → address bus
- Memory → instruction register (IR)
- PC incremented

Step 2: Decode

- Control unit interprets opcode
- Determines required operation

Step 3: Execute

- ALU performs operation
- Memory or registers updated

Step 4: Store (if needed)

- Result written back

Flow of Instruction Sequencing

PC → Fetch → Decode → Execute → Next PC

Branching in Instruction Sequencing

Execution order can change using:

(a) Conditional Branch

Executed only if condition is true.

Example:

JZ LABEL

Jump if zero.

(b) Unconditional Branch

Always jumps.

Example:

JMP 2000H

(c) Subroutine Call

- CALL → jumps to function
- RETURN → comes back

Addressing Modes and Assembly Language

Introduction

In computer architecture, the CPU must know **where the operand is located**.
Addressing modes specify how to find operands, while **assembly language** provides symbolic instructions that use these addressing modes.

Understanding both is essential for efficient programming and instruction execution.

Part A — Addressing Modes

Definition

Addressing mode is the method used by an instruction to specify the location of its operand.

It tells the CPU:

- Where the data is
- How to calculate the effective address
- How to fetch the operand

Need for Addressing Modes

- Provides programming flexibility
- Reduces instruction length
- Improves execution efficiency
- Supports complex data structures

- Enables relocation of programs

Types of Addressing Modes

1. Immediate Addressing Mode

Definition

Operand is present **inside the instruction itself**.

Format

MOV R1, #25

Explanation

- #25 is the actual data
- No memory access required for operand
- Fastest mode

Advantages

- Very fast
- Simple implementation
- No extra memory access

Disadvantages

- Limited operand size
- Not flexible

2. Register Addressing Mode

Definition

Operand is stored in a **CPU register**.

Example

ADD R1, R2

Explanation

- Both operands in registers
- No memory access needed

Advantages

- Very fast
- Short instruction length
- Efficient

Disadvantages

- Limited number of registers

3. Direct Addressing Mode

Definition

Instruction contains the **memory address of the operand**.

Example

LOAD R1, 2000H

Effective Address

EA = Address field of instruction

Advantages

- Simple
- Easy to understand

Disadvantages

- Limited address range
- One memory access required

4. Indirect Addressing Mode

Definition

The instruction specifies a memory location that contains the **address of the operand**.

Example

LOAD R1, (2000H)

Effective Address

EA = M[Address]

Advantages

- Allows large address space

- Flexible

Disadvantages

- Two memory accesses
- Slower than direct

5. Register Indirect Addressing

Definition

Register holds the address of the operand.

Example

LOAD R1, (R2)

Effective Address

EA = contents of R2

Advantages

- Faster than memory indirect
- Flexible

Disadvantages

- Needs extra register

6. Indexed Addressing Mode

Definition

Effective address is obtained by adding an index register to a base address.

Formula

EA = Base address + Index register

Example

LOAD R1, 1000H(R2)

Advantages

- Useful for arrays
- Supports loops

Disadvantages

- Slightly complex

7. Relative Addressing Mode

Definition

Effective address is computed relative to the program counter (PC).

Formula

EA = PC + Offset

Uses

- Branch instructions
- Position-independent code

Part B — Assembly Language

Definition

Assembly language is a low-level programming language that uses **mnemonics and symbolic addresses** to represent machine instructions.

It is machine dependent and closely related to hardware.

Characteristics

- Uses mnemonics (ADD, MOV, SUB)
- Uses symbolic addresses
- One-to-one mapping with machine code
- Requires an assembler
- Hardware dependent
- Faster execution than high-level languages

Structure of Assembly Language Instruction

Typical format:

[label] opcode operand(s) ; comment

Example

LOOP: ADD R1, R2 ; add R2 to R1

Components

1. Label

- Symbolic name for address
- Used in branching

2. Opcode

- Specifies operation
- Example: ADD, MOV

3. Operand

- Data or address
- May use addressing modes

4. Comment

- Explanation for programmer
- Ignored by assembler

Role of Assembler

Definition

An **assembler** converts assembly language into machine code.

Functions

- Translates mnemonics
- Assigns addresses
- Resolves labels
- Generates object code
- Reports errors

Advantages of Assembly Language

- Faster execution
- Efficient memory usage
- Hardware control
- Suitable for embedded systems
- Easier than machine code

Disadvantages

- Machine dependent
- Difficult to learn
- Time-consuming
- Poor portability
- Not suitable for large applications

Applications

- Embedded systems
- Device drivers
- Real-time systems
- System programming
- Performance-critical code